

Neural Network Project

Nima Saeidi

Getting Started

I followed the instructions in the notebook to get started. I didn't face any major problems while completing the initial functions.

My first **conceptual** difficulty was understanding **regularization**. After reading about it, things became clearer.

My first **programming** issue came when trying to **load the MNIST dataset**. After some research and trying different approaches, I was able to successfully write the functions to load the data.

Data Preprocessing

For preprocessing the data:

1. I normalized the input images to a range of 0 to 1.
 2. I performed **one-hot encoding** on the labels.
 3. I flattened the numpy arrays to make them compatible with the model.
-

Hyperparameter Tuning

I performed simple hyperparameter tuning using a **holdout method** and **grid search**.

However, after tuning, I encountered my biggest problem:

No matter what I did, my model's accuracy wouldn't exceed 20%!

Even with very large hidden layers (512, 256, 128, 64 neurons) and 1000 iterations, my accuracy stayed around 5%. (And the training took 2 hours 🤦.)

Debugging the Problem

I suspected that my **activation function** or **loss function** was wrong because getting 5% accuracy indicated something was fundamentally broken.

After further research, I found out that my theory was correct:

- Using **Mean Squared Error (MSE)** as a loss function is **terrible** for MNIST.
- Instead, I should use a **softmax activation function** in the output layer and a corresponding **cross-entropy loss**.

Improvements and Final Model

After fixing the loss and activation functions, I rewrote my functions, added **mini-batch training** with an **adaptive learning rate**, and achieved around **98% accuracy!** 🎯

List of Improvements

1. He (Kaiming) Initialization

- Used because we have **ReLU** as the activation function.
- Helps avoid **dead neurons** by using slightly larger initial weights.

 [Medium Guide](#)

 [Papers With Code](#)

2. Softmax Activation Function

Problems with Argmax:

- **Argmax** returns the index of the highest score, but its **gradient is zero**, making it useless for backpropagation.

What is Softmax?

- Softmax converts raw scores (logits) into probabilities.
- Ensures outputs are between 0 and 1 and that they **sum to 1**.

Why Softmax?

- Unlike **Sigmoid** or **ReLU**, **Softmax** is ideal for **multi-class classification** tasks.
- **Sigmoid** does not ensure outputs sum to 1.
- **ReLU** solves vanishing gradients but doesn't output probabilities.

 [Learn about Softmax](#)

 [More on Softmax](#)

3. Caching in Feedforward

- During feedforward, we compute intermediate values (A , Z).
 - These are **needed for backpropagation**.
 - By caching them, we avoid redundant calculations and speed up the model.
-

4. L2 Regularization

- Same technique I initially learned.
 - Helps prevent overfitting by penalizing large weights.
-

5. Mini-Batch Gradient Descent

Problems with Stochastic Gradient Descent (SGD):

- **High variance updates**: Noisy updates cause the model to jump around.
- **Slow convergence near optimum**: Harder to settle at the minimum.
- **Inefficient hardware utilization**: Modern hardware is better utilized with batches.

Solution: Mini-Batch Gradient Descent

- Groups data into small batches and performs gradient descent on them.
- Balances the benefits of full batch gradient descent and SGD.

- Faster and more stable convergence.
-

6. Dropout Regularization

What is Dropout?

- Dropout is a powerful regularization technique to prevent overfitting in neural networks.
- During training, randomly "drop" (set to zero) a percentage of neurons in each layer.
- This forces the network to learn more robust features that don't depend on specific neurons.
- Think of it as training multiple smaller networks and then averaging their predictions.

How Dropout Works:

1. During **forward propagation**:

- Randomly select a fraction of neurons to disable (set to zero)
- Scale the remaining neurons' outputs to maintain the same expected output

2. During **backpropagation**:

- Only update weights connected to "active" (not dropped) neurons
- Use the same dropout mask from forward propagation

3. During **testing/inference**:

- No neurons are dropped
- No scaling is needed (since all neurons are active)

Key Implementation Details:

- Added a `dropout_keep_prob` parameter (probability of keeping a neuron active)
- Created random dropout masks during forward propagation:

```
D = np.random.rand(*A.shape) < keep_prob

A = np.multiply(A, D)

A = A / keep_prob # Scale to maintain expected value
```

- Applied the same mask during backpropagation:

```
dA_prev = np.multiply(dA_prev, D) / keep_prob
```

- Disabled dropout during testing (is_training=False parameter)

Benefits of Dropout:

- Reduced overfitting and improved generalization
- Acts as an efficient form of "model averaging"
- Makes the network more robust by preventing co-adaptation of features
- Simple to implement with significant impact on performance

In Our Implementation:

- We added dropout with a keep probability of 0.8 (20% dropout rate)
- Applied dropout between hidden layers only (not on input or output layer)
- Ensured dropout is only active during training, not during evaluation or prediction

 [Original Dropout Paper by Hinton et al.](#)

 [Understanding Dropout](#)

7. Adam Optimizer (Adaptive Moment Estimation)

Problems with Standard Gradient Descent:

- Fixed learning rate requires manual tuning
- Slow convergence in complex landscapes
- Gets stuck in saddle points

Why Adam?

- Combines benefits of **Momentum** and **RMSprop**
- Adaptive learning rates for each parameter
- Faster convergence with less hyperparameter tuning

Key Features:

- Tracks exponentially decaying averages of gradients (m)
- Tracks squared gradients (v)
- Includes bias correction
- Default parameters work well ($\beta_1=0.9$, $\beta_2=0.999$)

 [Original Adam Paper](#)

Conclusion

By combining:

- **He initialization**
- **Softmax + Cross-Entropy Loss**
- **Mini-Batch Training**
- **Dropout & L2 Regularization**
- **Adam Optimization**

We achieved **98% accuracy** on MNIST - a massive improvement over the initial 15% with MSE loss!

 **Key Insight:** The combination of proper activation functions, loss functions, and optimization techniques is critical for success in deep learning.